

Talk Architecting Agentic Workflow

Wer kontrolliert den Agenten?

Der Bruch

- Wer hat diese Architektur-Entscheidung getroffen?

Ein Agent hat **14 Dateien** geändert und **committed**.
Antwort: **niemand** hat es freigegeben — das Modell hat gehandelt,
kein Mensch, kein Mechanismus.

The image shows a GitHub commit interface on the left and a flowchart on the right. The GitHub interface displays a commit by 'Agent (claude-3.5-sonnet)' with the message 'Refactor workflow and improve error handling'. It lists 14 files changed with a summary of +612 additions and -198 deletions. The flowchart on the right shows a process flow: 'AGENT' (represented by a robot icon) leads to 'PLAN', then 'CHANGE FILES', then 'COMMIT', and finally 'DONE.'. To the right of this flow, three boxes with red 'X' marks indicate missing controls: 'KEIN MENSCH hat freigegeben' (no human approval), 'KEIN MECHANISMUS hat geprüft oder gestoppt' (no mechanism for review or stop), and 'KEIN APPROVAL kein Review, kein Gate, keine Richtlinie' (no approval, no review, no gate, no guideline). A warning icon and text at the bottom state: 'NIEMAND HAT ES FREIGEgeben. Das Modell hat gehandelt. Kein Mensch. Kein Mechanismus. Keine Kontrolle.'

main ✓ Commit 8f3c2a1

Agent (claude-3.5-sonnet) committed 2 minutes ago
Message: Refactor workflow and improve error handling

14 files changed +612 -198

M	src/agents/runner.py	+45	-12	■■■
M	src/agents/tools.py	+78	-21	■■■
A	src/agents/prompts.py	+102	0	■■■
M	src/workflow/graph.py	+86	-15	■■■
M	src/workflow/state.py	+64	-10	■■■
A	src/workflow/edges.py	+93	0	■■■
M	src/execution/engine.py	+57	-18	■■■
M	src/execution/steps.py	+73	-22	■■■
A	src/execution/guards.py	+88	0	■■■
M	src/verification/checks.py	+66	-14	■■■
M	src/verification/asserts.py	+31	-6	■■■
A	src/governance/audit.py	+42	0	■■■
M	src/governance/logging.py	+36	-8	■■■
D	tests/test_old_runner.py	0	-62	■■■

+612 additions -198 deletions

AGENT

PLAN

CHANGE FILES

COMMIT

DONE.

KEIN MENSCH
hat freigegeben

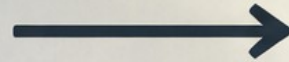
KEIN MECHANISMUS
hat geprüft oder gestoppt

KEIN APPROVAL
kein Review, kein Gate,
keine Richtlinie

⚠️ **NIEMAND HAT ES FREIGEgeben.**
Das Modell hat gehandelt.
Kein Mensch. Kein Mechanismus. Keine Kontrolle.

ARCHITEKTUR VERSCHIEBT SICH

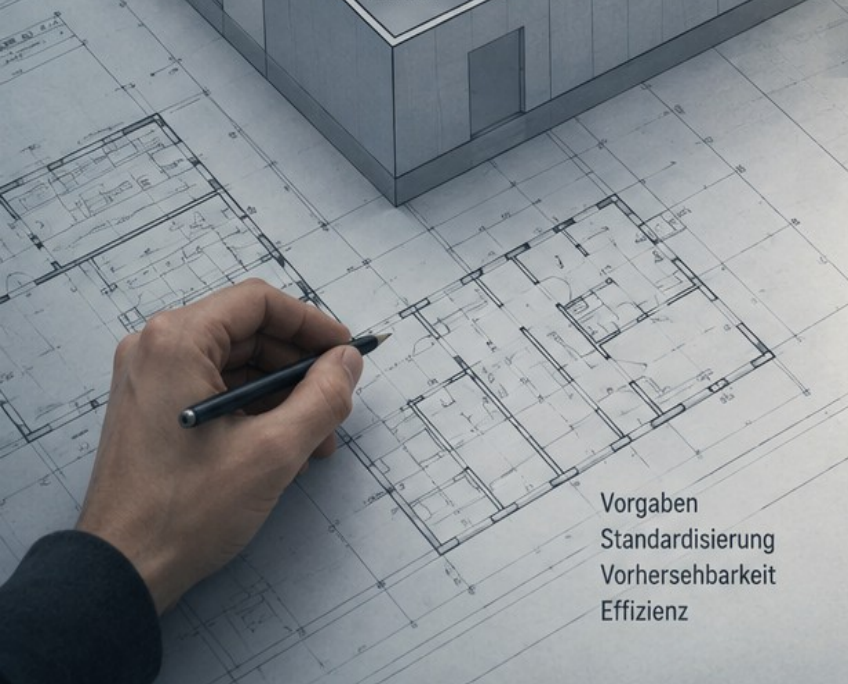
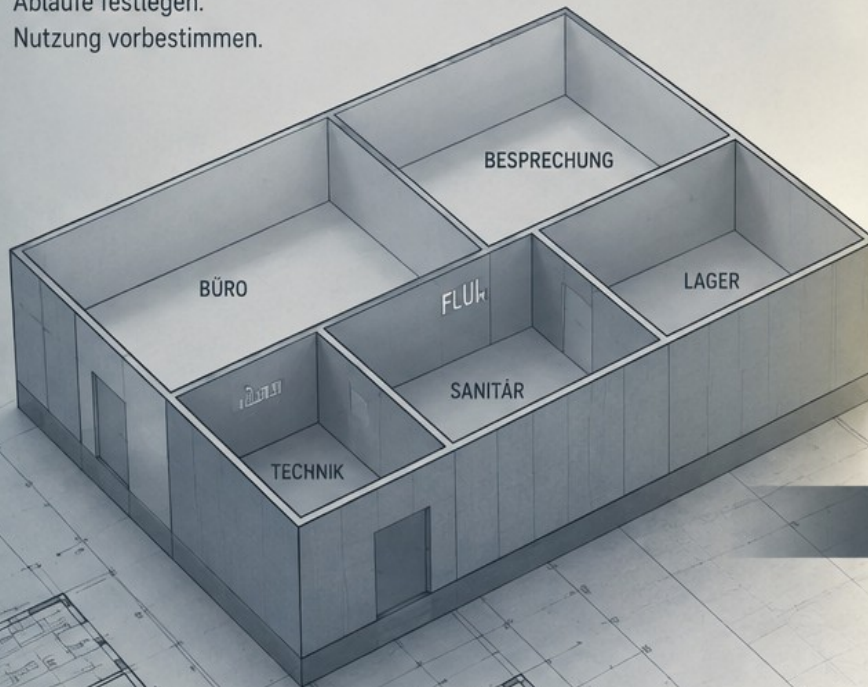
VOM ENTWURF DES PROGRAMMS



ZUM ENTWURF DES HANDLUNGSSPIELRAUMS.

ENTWURF DES PROGRAMMS

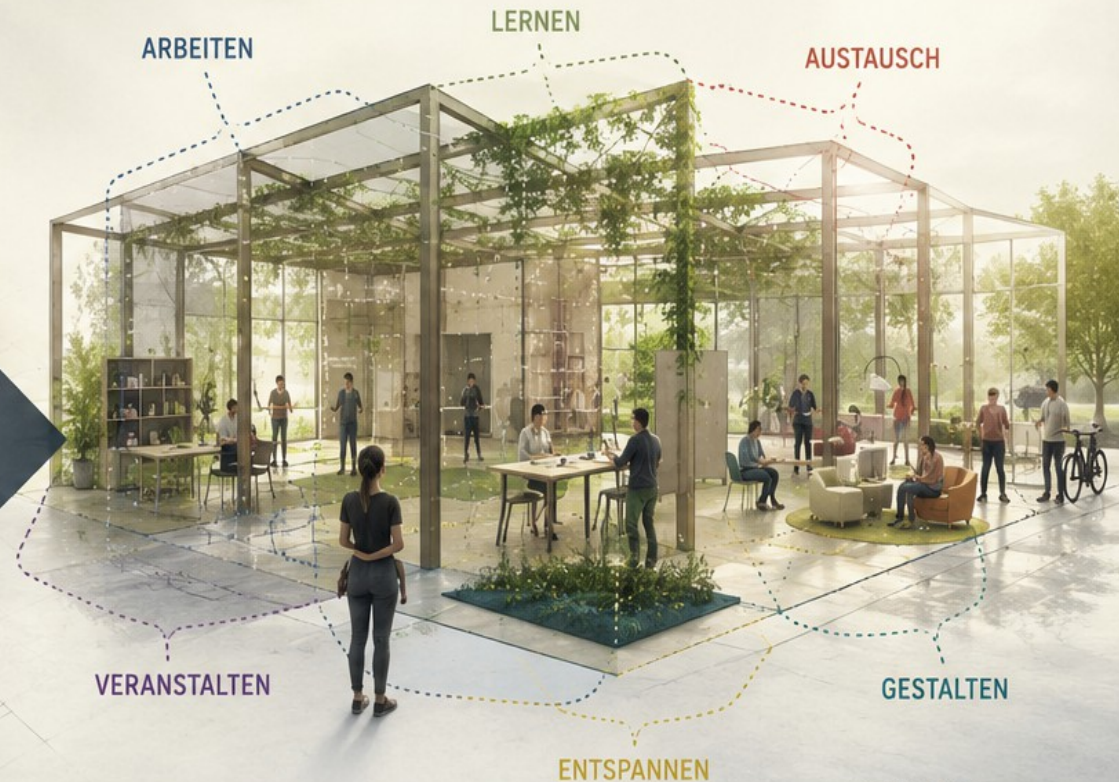
Funktion definieren.
Abläufe festlegen.
Nutzung vorbestimmen.



Vorgaben
Standardisierung
Vorhersehbarkeit
Effizienz

ENTWURF DES HANDLUNGSSPIELRAUMS

Möglichkeiten schaffen.
Nutzung ermöglichen.
Anpassung zulassen.



Freiheit
Anpassungsfähigkeit
Vielfalt
Resilienz

... UND VIELES MEHR

Wer kontrolliert den Agenten?

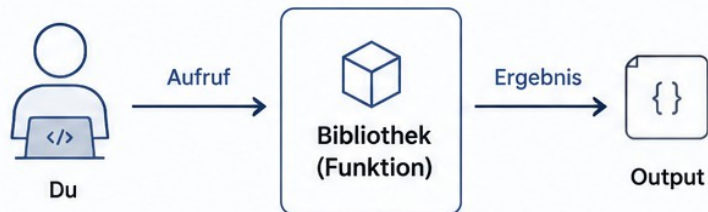
Eine Frage. Sieben Teil-Antworten — jede Folie beantwortet eine:

- Wer darf den Agenten stoppen? → Enforcement
- Wer hält den Zustand? → State
- Wer entscheidet den nächsten Schritt? → Control Surface
- Wer überprüft das Ergebnis? → Verification
- Wer kann Entscheidungen nachvollziehen? → Governance
- Wie bleiben parallele Agenten getrennt? → Isolation
- Wo bricht Kontrolle weg? → Failure Modes

Bibliothek vs. Agent

BIBLIOTHEK

Deterministisch. Du rufst auf.



🎮 Aufruf	du rufst auf
🎯 Verhalten	deterministisch
🛡️ Kontrolle	der Aufrufer
✅ Erwartung	vorhersagbar
🔧 Zweck	Werkzeug für deinen Code

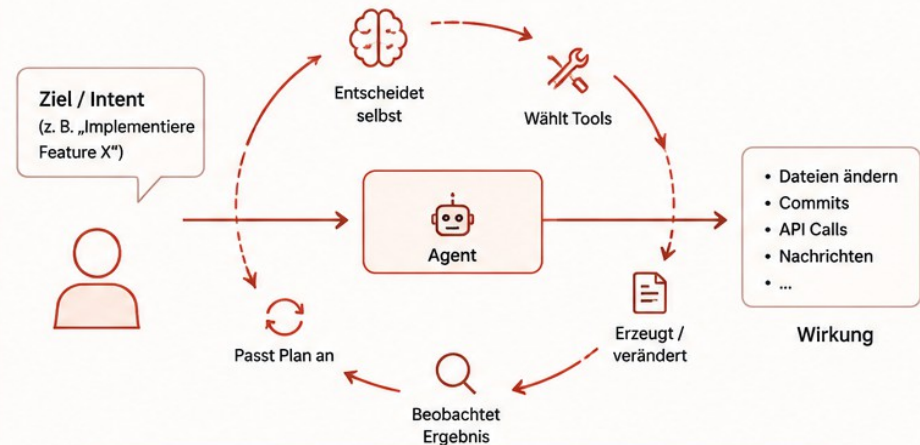


Klare Eingabe. Klare Ausgabe.
Du behältst die Kontrolle.

VS.

AGENT

Nicht-deterministisch. Er handelt selbst.



👤 Aufruf	du gibst ein Ziel – er startet
🔄 Verhalten	nicht-deterministisch
❓ Kontrolle	offen – genau das Problem
📈 Erwartung	variabel, abhängig von Kontext & Lauf
🤖 Zweck	eigenständige Ausführung komplexer Aufgaben



Eigenständiger Kontrollfluss.
Du definierst den Handlungsspielraum – nicht jeden Schritt.



Kernunterschied: Eine Bibliothek führt aus, was du sagst.
Ein Agent entscheidet, was er als Nächstes tun wird.

„Agenten sind keine Bibliotheken.
Sie sind laufende Systeme mit eigenem Kontrollfluss.“

Enforcement — das Problem

Der Agent tut, was er nicht darf

- Der Prompt sagt: „kein Code ohne freigegebenen Plan“
- Der Agent schreibt trotzdem
- Ein Prompt ist eine Bitte — kein Vertrag

Enforcement — die Entscheidung

Regeln gehören ausserhalb des Modells

- Eine Regel, die das Modell brechen kann, ist keine Regel
- Der Mechanismus, der „nein“ sagt, darf nicht im selben nicht-deterministischen Layer leben
- Enforcement liegt ausserhalb der Prompt-Schicht

Enforcement — Trade-offs

Was kostet das?

- Verworfen: Regeln nur im System-Prompt — das Modell kann sie ignorieren
- Verworfen: Fine-Tuning auf Regeltreue — teuer, nie 100 %, nicht auditierbar
- Gewählt: harter Mechanismus (Hook / Guard)
- Preis: weniger Flexibilität, mehr Reibung — Ausnahmen müssen explizit modelliert werden
- Anders entscheiden: reine Read-/Explore-Agenten ohne Seiteneffekte → Prompt-Disziplin reicht

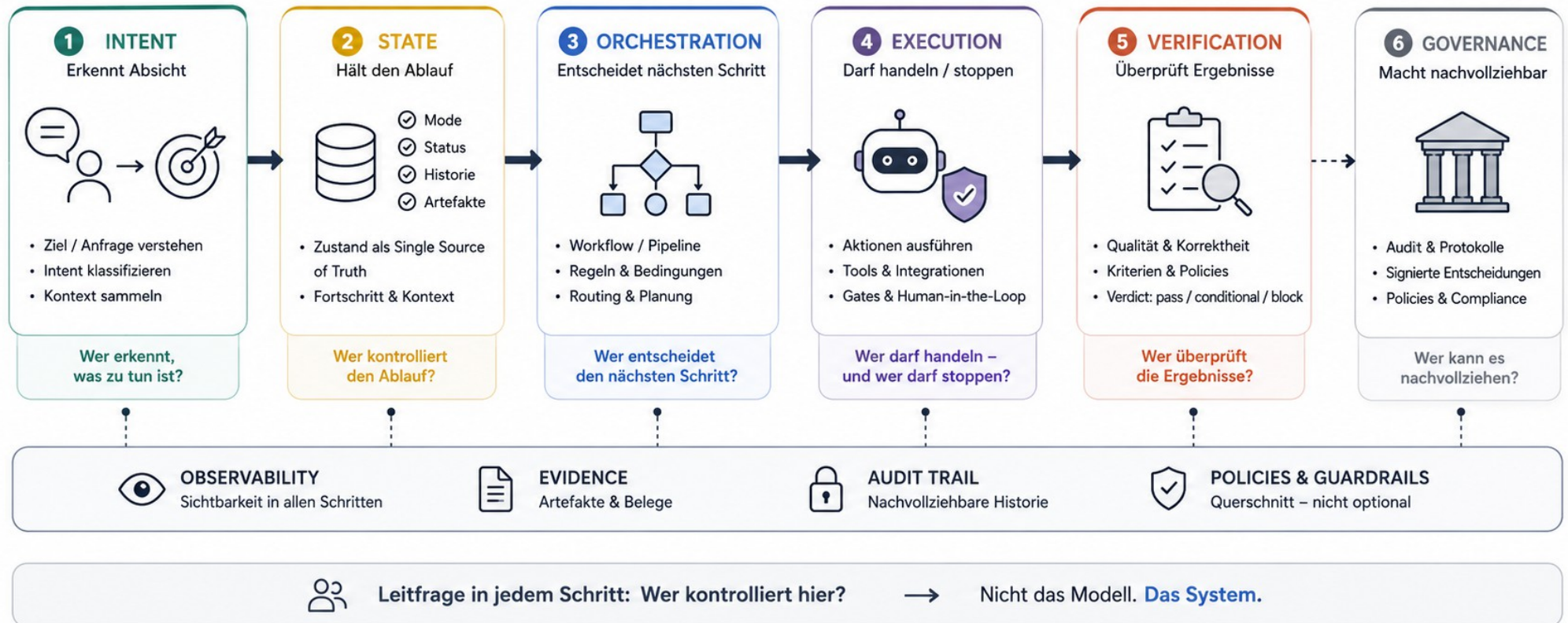
Demo 1 - Enforcement

Wie erzwingt man Regeln ausserhalb des Modells?

- Agent will schreiben → Kontrollpunkt blockt → Agent wird umgeleitet
- Fokus: der Vertrag — Auslöser · Bedingung · Verdict
- Austauschbar mit: LangGraph-Edge-Guard, OpenAI-Tool-Gating, eigenem Middleware-Layer

Agentic Workflow – Die Architektur des Kontrollflusses

Wer kontrolliert den Agenten?



State – das Problem

Der Agent verliert den Faden

- Er vergisst, wo er steht
- Und dreht sich im Kreis

Warum es bricht

- Der Zustand steckt implizit im Prompt-Verlauf
- Unsichtbar — niemand kann ihn auslesen
- Unzuverlässig — Kontext fällt aus dem Fenster
- Nicht unterbrechbar — kein definierter Pausenpunkt

State — Entscheidung & Trade-offs

Expliziter, externer Zustand

- Entscheidung: eine State Machine mit wenigen, benannten Modi — außerhalb des Modells
- Verworfen: impliziter Kontext — unsichtbar, flüchtig
- Verworfen: Modell merkt sich den Fortschritt — nicht inspizierbar, nicht reproduzierbar
- Verworfen: volle Workflow-Engine (BPMN) — überschwer für Agenten-Granularität
- Preis: starrer; jeder Modus muss explizit modelliert werden
- Anders entscheiden: einzelner, kurzlebiger Task ohne Wiederaufnahme

State — Umsetzung

Wer den Zustand hält, kontrolliert den Ablauf

- Wenige benannte Modi mit definierten Übergängen
- Austauschbar mit: LangGraph-State, Enum + Reducer, Status-Feld in der DB
- Auch ohne dieses Repo: hält das Modell den Zustand, kontrolliert es sich selbst

Brücke

Zwei offene Fragen

- Wer wählt den nächsten Schritt? — der Workflow oder das Modell?
- Wer darf handeln, und wer darf stoppen? — die Mechanik oder der Mensch?

Beide Fragen führen zur selben Architekturentscheidung: der Control Surface.

Control Surface — das Problem

Determinismus oder Autonomie — beides hat einen Preis

Skriptgesteuerte Pipeline

- Vorhersagbar
- Aber blind für Unerwartetes

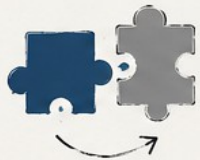
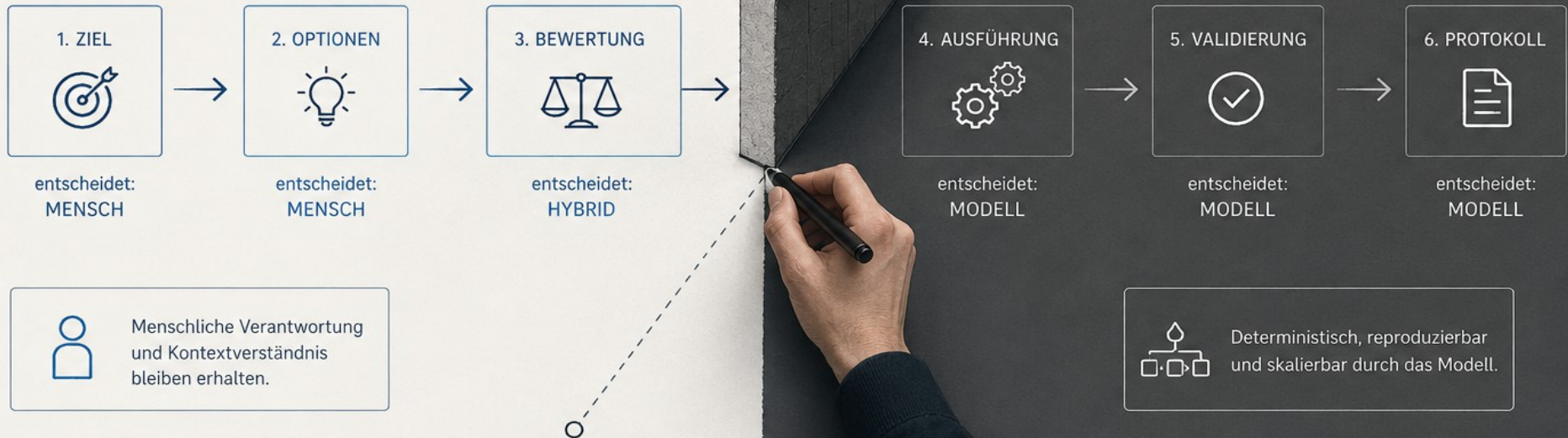
Modellgetriebene Autonomie

- Flexibel
- Aber driftet, nicht reproduzierbar

Beide Extreme als globaler Default sind falsch.

Architektur ist, wo du die Grenze ziehst

Pro Schritt deklariert, wer entscheidet



Austauschbar mit:

jeder Orchestrierung, die deterministische und model-driven Stages mischt

Auch ohne dieses Repo:

wer die Grenze nicht zieht, hat sie zugunsten des Modells gezogen.



Demo 2 - Wer entscheidet

Wer trifft Entscheidungen?

- Derselbe Task zweimal: orchestriert (Workflow) vs. model-driven (Modell)
- Die deterministische Variante ist reproduzierbar — die andere driftet

Verification

„Sieht gut aus“ ist nicht automatisierbar

- Failure Mode: der Verifier antwortet in Freitext — eine Maschine kann damit nichts tun
- Warum es bricht: Freitext ist nicht maschinell verzweigbar
- Entscheidung: strukturiertes Verdict nach festem Schema — pass / conditional / block
- Verworfen: Freitext-Review — nicht gate-fähig
- Verworfen: reiner Boolean (pass/fail) — verliert die Stufe „mit Auflagen“
- Preis: ein Schema verliert Nuance und muss gepflegt werden
- Anders entscheiden: reines Human-Review ohne nachgelagerte Automatik

Demo 3 - Verifikation

Demo · Wie kapselt man Verifikation?

- Ein deklarierter Gatekeeper prüft ein Artefakt
- Ergebnis: strukturiertes Verdict statt Fließtext — deklariert, nicht programmiert
- Austauschbar mit: jedem Tool mit typisiertem Output (JSON-Schema, Pydantic, Enum)
- Prinzip: Verdict als Datenstruktur, nicht als Text

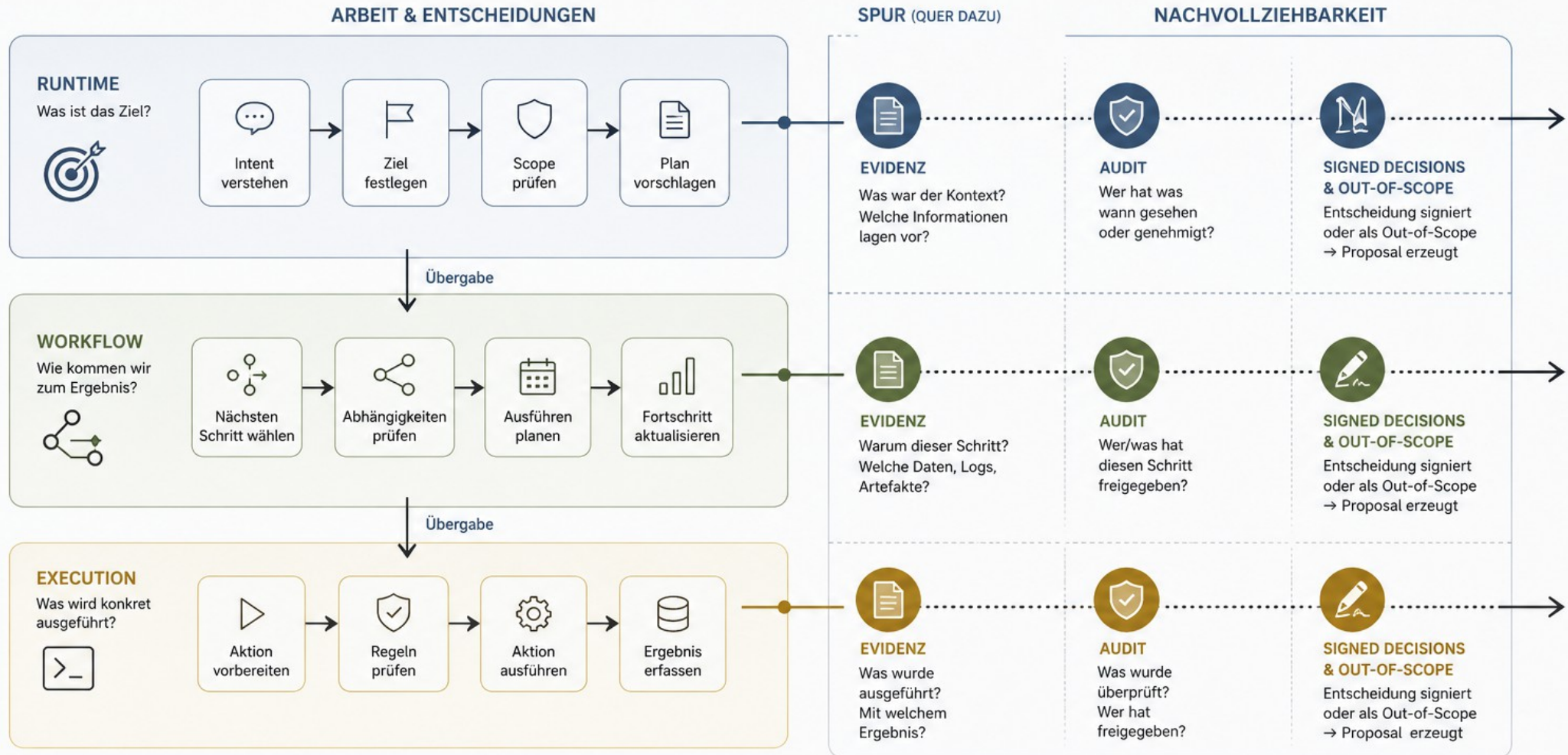
Governance — das Problem

„Warum hat der Agent das getan?“

- Drei Wochen später fragt jemand — und niemand kann es rekonstruieren
- Audit nachträglich anhängen scheitert: die Begründung ist zum Entscheidungszeitpunkt schon verloren
- Governance ist kein Feature, das man anhängt

EINE SPUR IN JEDER SCHICHT

Eine Spur in jeder Schicht — quer dazu — Evidence · Audit · Signed Decisions



DIE ARCHITEKTUR ERZWINGT DIE SPUR
Nicht das Gewissen des Entwicklers.



Unveränderbar
(append-only)



Vollständig
und zeitgestempelt



Rollenbasiert
zugreifbar



Durchsuchbar
und exportierbar



Ablauf innerhalb der Schicht



Spur quer durch alle Schichten



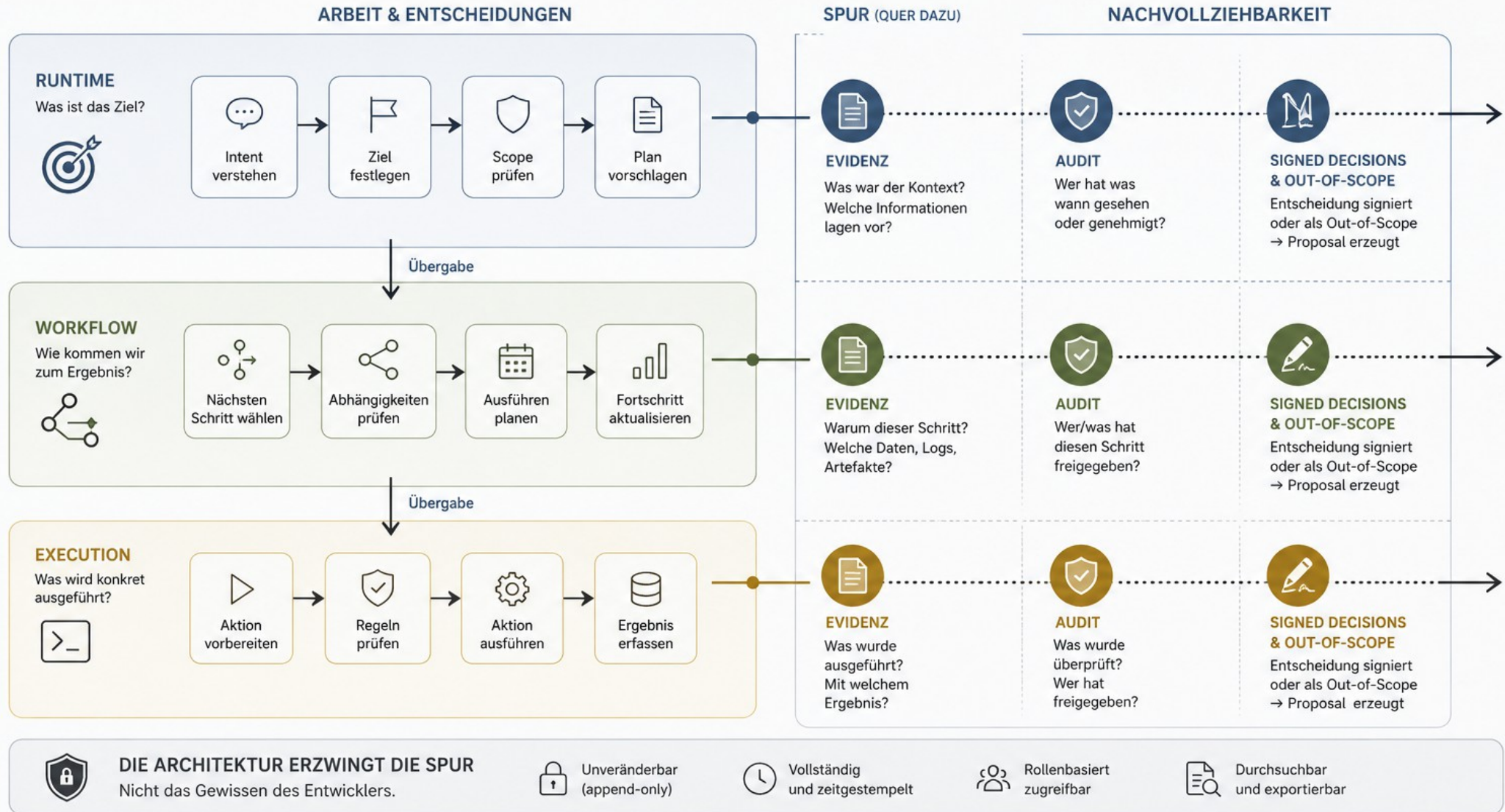
Verbindet Arbeit mit Nachvollziehbarkeit



Out-of-scope? → Proposal statt Aktion.
Keine blinden Ausführungen.

EINE SPUR IN JEDER SCHICHT

Eine Spur in jeder Schicht — quer dazu — Evidence · Audit · Signed Decisions



→ Ablauf innerhalb der Schicht

..... Spur quer durch alle Schichten

— Verbindet Arbeit mit Nachvollziehbarkeit



Out-of-scope? → Proposal statt Aktion.
Keine blinden Ausführungen.

Governance — Trade-offs

Was kostet die Spur?

- Verworfen: Audit nachträglich aus Logs — Begründung & Freigabe-Kontext sind dann weg
- Verworfen: Governance als separate Schicht obendrauf — umgehbar, der Kernpfad kennt es nicht
- Gewählt: Spur in jede Schicht eingebaut
- Preis: mehr Pflicht-Artefakte, mehr Reibung pro Schritt
- Anders entscheiden: Wegwerf-Prototypen und Spikes

Failure Modes

Wo Kontrolle wegbricht

- Gates nur im Prompt → der Agent schreibt trotzdem
- Kein expliziter Zustand → Endlosschleifen / Drift
- Kein Audit → nicht nachvollziehbar
- Gemeinsamer Workspace → Race Conditions
- Freitext-Verdicts → nicht automatisierbar
- Kein Kill-Kriterium → Sunk Cost, kein Ausstieg
- Control Surface nie gezogen → das Modell entscheidet alles, implizit

Jede Zeile = eine nicht getroffene Entscheidung.

Die Architekten-Checkliste

Sieben Fragen an die Architektur

- Welche Komponente besitzt den Zustand?
- Welche Entscheidungen sind ans Modell delegiert — welche an die Mechanik?
- Welche Regeln sind mechanisch erzwingbar — und liegen sie außerhalb des Modells?
- Wo verläuft deine Control Surface — pro Teilschritt?
- Liefert deine Verifikation ein maschinenlesbares Verdict?
- Wo entstehen Audit-Artefakte — automatisch oder gar nicht?
- Wie werden parallele Agenten isoliert?

Architektur für Agenten heisst: den Handlungsspielraum entwerfen — und Kontrolle mechanisch erzwingen.

Wer die Control Surface nicht zieht, hat sie trotzdem gezogen — zugunsten des Modells.

ENDE